Part Three

Tyler Dinh[1], Kevin Tang[1], and Demi Babalola[1]

[1]Washington University in St. Louis

October 16, 2026

1 Algorithmic Design and Implementation Details

The objective function for this project was Mean Squared Error (MSE), defined as $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$.

Then, the gradient equation used for parameter updates for step i can be calculated as:

$$w^{i+1} = w^i - \alpha\frac{\delta MSE}{\delta m}$$

$$\frac{\delta MSE}{\delta m} = \frac{-2}{n}\sum_{i=1}^{n}(y_i - (mx_i + b))$$

Algorithm 1 Custom Gradient Descent Implementation

procedure GRADIENT DESCENT(W^0, b^0, X, y, η, N)
  for $i = 0$ to N **do**
    $\text{err} = f(X, W^i, b^i) - y$
    $W^{i+1} = W^i - \eta \cdot \frac{2}{|X|} X^\top \text{err}$
    $b^{i+1} = b^i - \eta \cdot 2\,\overline{\text{err}}$
  end for
  return W^N, b^N
end procedure

To implement this, first, we designated rows 501–630 as the testing set (130 samples) and the remaining 900 samples as the training set, and split into a features matrix and output vector for each set. Model parameters were never updated based on the test dataset. We then set a list of learning rates for the base gradient descent implementation as α = [1e-9, 1e-8, 1e-7], initial coefficient weights matrix to be all zeroes, initial bias to be zero, and iterations to be 1000. In terms of the algorithm's stopping criterion, our implementation automatically stops after $i = 1000$ completes, or in other words finishes its 1000th iteration. The solution efficiently calculates the required values of the coefficient weights matrix, the bias, MSE, and R^2 by looping over our learning rates.

Our dependencies for our project were 'ucimlrepo==0.0.7', 'pandas==3.0.5', 'matplotlib==3.11.2', 'numpy==2.5.3', 'scikit-learn==1.9.1', 'statsmodels==0.15.0', and 'python==3.12'. All dependencies are pinned and can be installed via 'uv' as described in the last section of this report.

2 Feature Scaling and Gradient Descent Dynamics

The two implementations for the unscaled and scaled features for the multivariate model used the same features, the same train/test split, the same zero-initialized weights, the same 1000 iterations, and the same gradient descent functions. In the implementation for the unscaled features multivariate model, raw features are used directly, and since their magnitudes vary quite wildly, the loss surface becomes quite sharp. As such, we chose to use extremely tiny learning rates ($1e-9$ to $1e-7$) so that the curve would converge. In the implementation for the scaled features multivariate model, the features matrix is standardized using the training set's mean and standard deviation, which makes the loss function much smoother. As a result, we use much larger learning rates ($1e-4$ to $1e-2$) which lead to convergence of gradient descent more smoothly.

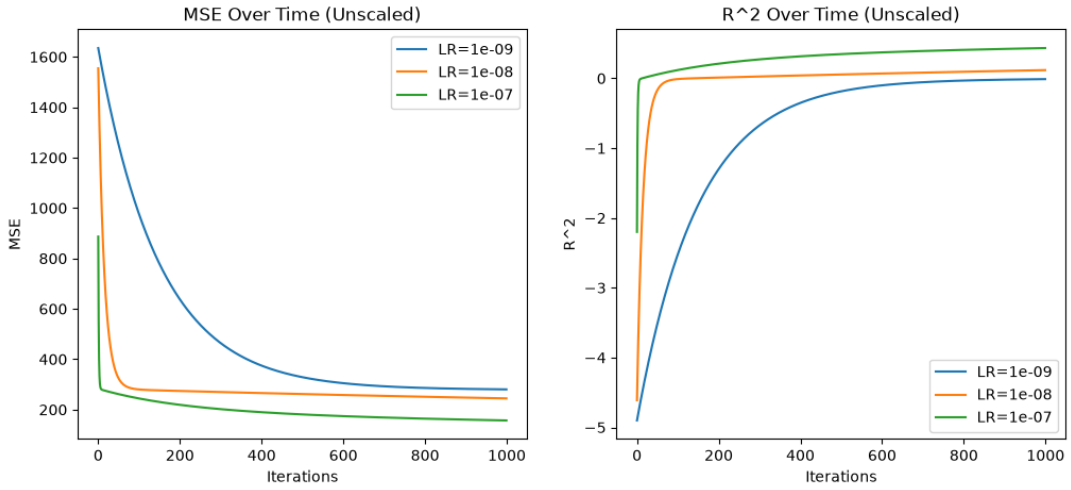


Figure 1: The unscaled multivariate model and its MSE and R^2.

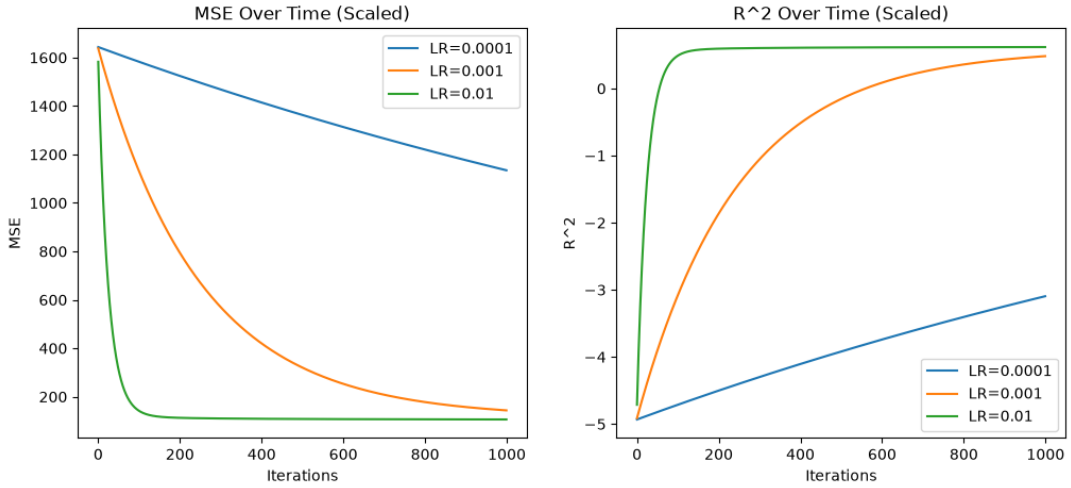


Figure 2: The scaled multivariate model and its MSE and R^2.

Comparing Figure 1 and Figure 2, we find that our algorithm performs better on the test set (lower MSE and a higher R^2) when we scale the features compared to when we do not scale the features.

Finally, we used sklearn to find the optimal solution for an unscaled multivariate model and a scaled multivariate model. This was implemented by fitting two sklearn LinearRegression models, with one on the unscaled training feature matrix, and the other on the scaled training feature matrix, and calculating MSE and R^2 for both.

	Unscaled Multivariate Model	**Scaled Multivariate Model**
MSE	141.258	141.258
R^2	0.351	0.351

Table 1: Comparison of a sklearn unscaled multivariate model with a sklearn scaled multivariate model.

As shown in Table 1, scaling does not meaningfully change the optimal mathematical solution. Although the weight values are different for a scaled model, the output of the scaled model on scaled inputs is identical to the output of the unscaled model on unscaled inputs. We observe that the MSE and R^2 on the test set for both a scaled and unscaled model are identical, at 141.258 and 0.351 respectively.

3 Linear vs. Non-Linear Feature Transformations

A logarithmic transformation, $\log(x+1)$, was applied to the base feature matrix for the training and test splits and a new multivariate model was trained on the custom gradient descent implementation.

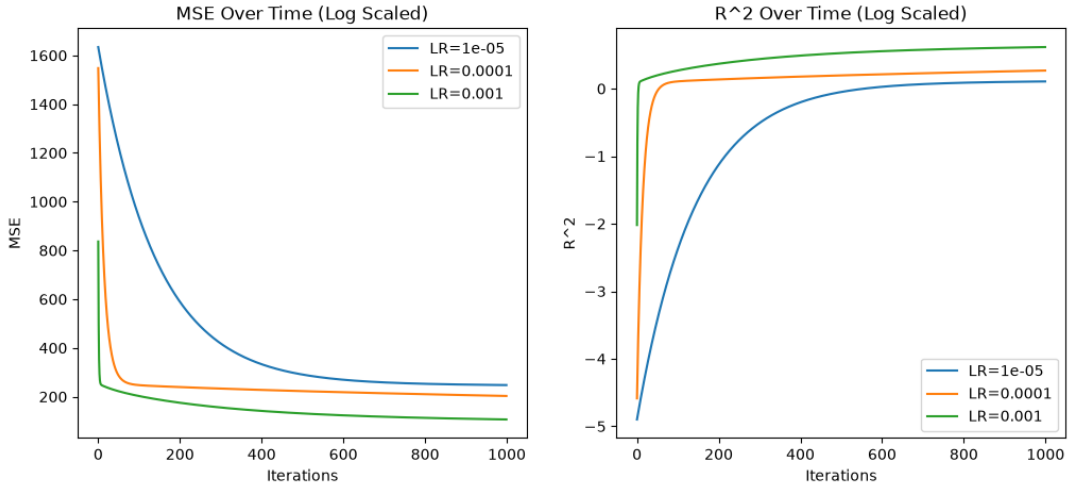


Figure 3: The logarithmically transformed multivariate model and its MSE and R^2.

The log (non-linear) transformation increases convergence speed and stability greatly over the base features as shown in Figure 3. As such, we were able to use much higher learning rates without the model diverging. By compressing large values from the base features via the transformation, it reduces the influence and presence of outliers, so that gradient descent can proceed much more smoothly.

	Untransformed	**Standardized**	**Log-Transformed**
Training MSE	104.154	104.154	55.831
Testing MSE	141.258	141.258	54.520
Training R^2	0.624	0.624	0.799
Testing R^2	0.351	0.351	0.750

Table 2: Comparison of an untransformed, standardized, and log-transformed multivariate model.

As shown in Table 2, while the untransformed and normalized models perform the same on the testing and training sets, the log (non-linear) transform improved the optimal solution and overall predictive performance. This is because the log transformation helps with features that don't have a linear relationship with the target feature, as well as making right-skewed distributions more normal by stretching out low-valued data and compressing high-valued data, which reduces heteroscedasticity.

4 Univariate vs. Multivariate Model Dyanamics

We trained eight seperate univariate models in the structure of $y = m_i x_i + b$ for every untransformed predictor until we achieved a positive R^2 value for two of the eight models. This was accomplished by looping over each feature in the set of features, partitioning the feature matrix into the train and test splits for the given feature, and then running our custom gradient descent function. We used initial weights of zero, initial bias of zero, a cap on iterations being 10,000, and a list of learning rates for α = [1e-7, 1e-6, 1e-5], and plotted the MSE and R^2 for each univariate model.

The easiest models to train were for all mix types, as they quickly converged to a positive R^2 during training with the learning rates we tested. The most difficult models were Superplasticizer, Coarse Aggregate, and Fine Aggregate, which we were unable to get to converge properly at our used learning rates, and if we increased the learning rates too much, the model diverged and output an overflow error or NaN.

	Univariate Coefficient	**Multivariate Coefficient**
Cement	0.133	0.081
Blast Furnace Slag	0.125	0.031
Fly Ash	0.107	-0.053
Water	-0.133	-0.206
Superplasticizer	0.116	0.883
Coarse Aggregate	0.029	-0.027
Fine Aggregate	0.034	-0.031
Age	0.119	0.095

Table 3: Comparison of coefficients from univariate models to a multivariate model.

We use sklearn to find the optimal solution for each of the univariate models, and compare the individual coefficients from these univariate models to the corresponding feature weights found previously via sklearn for the optimal untransformed multivariate model. We observe in Table 3 that the coefficients for the models involving individual features are different from the corresponding coefficient in the multivariate model. For example, perhaps the largest differences are that Fly Ash, Coarse Aggregate, and Fine Aggregate have a positive correlation with the target feature in the multivariate model while they have a negative correlation in their respective univariate models.

We also observe that the univariate coefficients are not equal to the multivariate coefficients because some features in our dataset may correlate with one another. Therefore, when training the univariate model for a single feature such as Fly Ash, the single coefficient will also account for the effects of all other features that are correlated to Fly Ash, which is omitted variable bias. When we train the multivariate model however, weights will be assigned based on only the contribution of that feature to achieve the lowest possible error, since the overall model now includes all features, eliminating omitted variable bias.

5 Statistical Inference and Regression Analysis

We use statsmodels.api.OLS to extract p-values for all eight features in the dataset across multiple models. This is implemented with untransformed features, standardized features, and log-transformed features, where each has an added constant term. The added constant term is necessary due to the fact that statsmodels.api.OLS does not include an intercept by default. Omitting the constant assumes the regression line formed by OLS passes through zero, which incorrectly changes the estimates for each of the features, and so the constant is needed.

	Untransformed	**Standardized**	**Log-Transformed**
Cement	6.971e-41	6.971e-41	1.299e-86
Blast Furnace Slag	4.145e-26	4.145e-26	4.519e-36
Fly Ash R^2	1.895e-13	1.895e-13	2.605e-01
Water R^2	1.918e-03	1.918e-03	1.863e-20
Superplasticizer R^2	2.461e-01	2.461e-01	1.040e-05
Coarse Aggregate R^2	5.443e-03	5.443e-03	4.079e-01
Fine Aggregate R^2	2.738e-03	2.738e-03	9.477e-02
Age R^2	9.516e-74	9.516e-74	1.747e-192

Table 4: Comparison of feature p-values for an untransformed, standardized, and log-transformed multivariate model.

There are trends between which predictors are important, and which univariate models were easier to train, as shown in Table 4. For example, a lower p-value meant the univariate model corresponding to that feature was easier to train. We found that the Cement mix type was the univariate model that converged the most quickly and smoothly, agreeing with its small p-values, while we had some more trouble getting Superplasticizer, Coarse Aggregate, and Fine Aggregate to converge properly at the learning rates we tried, which makes sense, looking at their relatively large p-values.

Interestingly, we observe log transformation changes the p-values significantly, while the p-values of the normalized model are exactly the same as without feature scaling. This is because normalization is a linear transformation, so it will not affect the underlying shape of the distribution of any of the features, and the t-statistic remains the same as before mathematically. However, with log scaling, it is not linear, and the shapes of the distributions are not preserved so the p-value can change, decreasing especially in instances where the original data was not linear/skewed and made more linear by the transformation.

6 Code and Reproducibility Access

The repository containing all of our implementation can be found at this GitHub repository, which should be public and cloneable.

To run the implementation, please install 'uv' locally and run the command 'uv sync', which will locally install all dependencies for the project into a virtual environment. Next, you must be in the created virtual environment to run any of the code, which can be activated via 'source .venv/bin/activate'. Then, click 'run all cells' within the 'main.ipynb' file to run all code. Nothing else is required.